Prompts y evidencias taller 2

Prompt: el recuadro marca el inicio y el fin.	Evidencia: el commit queda resaltado.	Imagen: espacio reservado para la captura.

01 PROMPT INICIAL

Quiero desarrollar una primera prueba de concepto (PoC) de un Data Privacy Vault, cuyo objetivo es proteger información personal identificable (PII – Personally Identifiable Information) mediante mecanismos de detección, tokenización y sustitución de datos sensibles.

La solución debe permitir recibir mensajes de texto que contengan información personal, identificar los datos sensibles y reemplazarlos por tokens alfanuméricos, evitando exponer la información original en los sistemas que consuman posteriormente el mensaje.

El proyecto debe comenzar con una implementación sencilla y funcional, pero su arquitectura debe estar preparada para evolucionar hacia una solución escalable, segura y mantenible.

No quiero una implementación monolítica ni código acoplado que dificulte incorporar nuevas funcionalidades en el futuro.

Para esta primera iteración, necesito implementar un único endpoint:

`POST /anonymize`

El endpoint recibirá un objeto JSON con una propiedad message, que contendrá una cadena de texto con posibles datos personales.

Inicialmente, el sistema deberá identificar y tokenizar los siguientes tipos de PII:

- Nombres completos de personas.
- Direcciones de correo electrónico.
- Números de teléfono.

El sistema deberá reemplazar cada dato identificado por un token alfanumérico, manteniendo intacta la estructura y el contenido restante del mensaje.

Ejemplo de petición

```
curl -X POST http://localhost:3001/anonymize \
-H "Content-Type: application/json" \
-d '{
  "message": "Oferta de trabajo para Dago Borda con email dborda@gmail.com y teléfono
3152319157"
}'
```

01 PROMPT INICIAL

Ejemplo de respuesta esperada

```
{
  "anonymizedMessage": "Oferta de trabajo para a81f9c20 con email b09bde30 y teléfono
dd195b6c"
}
```

Los tokens anteriores son ilustrativos. La implementación debe generar tokens mediante mecanismos criptográficamente seguros.

Reglas de tokenización

- Cada dato sensible debe recibir un token alfanumérico.
- El token no debe derivarse directamente del dato original ni permitir deducirlo.
- Si un mismo dato aparece varias veces dentro del mensaje, debe utilizar el mismo token durante el procesamiento de esa solicitud.
- Los diferentes tipos de PII deben manejarse de forma diferenciada.
- El sistema debe estar preparado para conservar la relación entre el dato original y su token, permitiendo implementar posteriormente mecanismos de desanonimización controlada.
- La generación y gestión de tokens debe estar desacoplada del mecanismo de detección de PII.

> **Importante: La sustitución mediante tokens es un mecanismo de seudonimización, no necesariamente de anonimización irreversible. La arquitectura debe contemplar esta distinción.**

Requisitos de arquitectura

Quiero implementar el proyecto utilizando principios de Clean Architecture, separación de responsabilidades y bajo acoplamiento.

Evalúa y utiliza una estructura basada en las siguientes capas:

Domain

Contendrá las entidades, objetos de valor, reglas de negocio y contratos fundamentales del dominio.

No debe depender de frameworks, bases de datos ni servicios externos.

Application

Contendrá los casos de uso y la orquestación de los procesos de negocio.

Por ejemplo:

- AnonymizeMessageUseCase.
- DetectPIIUseCase.
- GenerateTokenUseCase.

Utiliza interfaces o contratos para definir las dependencias con servicios externos.

01 PROMPT INICIAL

Infrastructure

Contendrá las implementaciones concretas de las interfaces:

- Servicio de detección de PII.
- Generador criptográficamente seguro de tokens.
- Repositorio para almacenar la relación entre PII y tokens.
- Adaptadores para futuras integraciones con bases de datos o proveedores externos.

Presentation

Contendrá los controladores HTTP, rutas, validaciones de entrada y configuración de respuestas. Mantén los controladores delgados: no deben contener lógica de negocio.

Principios adicionales

- SOLID.
- Dependency Inversion.
- Separation of Concerns.
- DRY, KISS y Clean Code.
- Inyección de dependencias.
- Programación orientada a interfaces.
- Alta cohesión y bajo acoplamiento.

No introduzcas abstracciones innecesarias. Cada interfaz debe tener una responsabilidad clara y responder a una necesidad real de extensibilidad o desacoplamiento.

Requisitos de tecnología

- Utiliza las siguientes tecnologías:
- Node.js como entorno de ejecución.
- TypeScript para aprovechar el tipado estático y mejorar la mantenibilidad.
- Express.js o una alternativa ligera que justifiques.
- NPM para la gestión de dependencias.
- Jest o Vitest para pruebas automatizadas.
- Selecciona versiones estables y compatibles entre sí.Prioriza las APIs nativas de Node.js cuando sean suficientes, especialmente para operaciones criptográficas.

EVIDENCIA EN COMMIT DEL REPOSITORIO `2e46e5a99b06b81d84611a213756eca9fccb34e4` *(Add initial project structure with data privacy vault functionality)*

02 SEGUNDO PROMPT (DESANONIMIZACION)

Perfecto, ahora vamos a implementar un nuevo endpoint:

`POST /deanonymize`

Este endpoint recibirá un mensaje que contiene tokens generados previamente por el endpoint /anonymize. El sistema deberá identificar los tokens presentes en el mensaje, consultar sus asociaciones almacenadas en el Data Privacy Vault y reemplazarlos por los valores originales correspondientes.

El resultado debe conservar el contenido, formato y estructura del mensaje original, modificando únicamente los tokens reconocidos.

Ejemplo de petición

```
curl -X POST http://localhost:3001/deanonymize \
-H "Content-Type: application/json" \
-d '{
  "anonymizedMessage": "oferta de trabajo para NAME_e1be92e2b3a5 con email EMAIL_8004719c6ea5
y telefono PHONE_40e83067b9cb"
}'
```

Ejemplo de respuesta esperada

```
{
  "message": "oferta de trabajo para Dago Borda con email dborda@gmail.com y telefono
3152319157"
}
```

Los tokens del ejemplo son ilustrativos. La implementación debe trabajar con las asociaciones reales generadas y almacenadas durante el proceso de anonimización.

Requisitos funcionales

- **Recuperación de token**
- El sistema debe identificar los tokens presentes en anonymizedMessage.
- Inicialmente debemos soportar los siguientes prefijos:
  - NAME_: nombres completos.
  - EMAIL_: direcciones de correo electrónico.
  - PHONE_: números telefónicos.
- La solución debe ser extensible para soportar nuevos tipos de PII sin modificar la lógica central del caso de uso.
- **Recuperación de información original**

Para cada token identificado:

- Consultar el repositorio de tokens existente.
- Verificar que el token exista.
- Recuperar el valor original asociado.

02 SEGUNDO PROMPT (DESANONIMIZACION)

- Sustituir el token por su valor original.
- Mantener intacto el resto del mensaje.
- No se deben generar nuevos tokens ni reconstruir valores mediante algoritmos de transformación.
- La recuperación debe realizarse exclusivamente utilizando las asociaciones almacenadas durante la anonimización.

Consistencia

- Un mismo token debe recuperar siempre el mismo valor original dentro de su ámbito de seguridad.
- Si un token aparece varias veces en el mensaje, todas sus apariciones deben sustituirse.
- Los tokens deben procesarse sin alterar accidentalmente otras partes del texto.
- No deben producirse sustituciones parciales de tokens.
- Los valores originales pueden contener caracteres especiales y espacios, que deben recuperarse correctamente.
- El proceso debe respetar el orden y contenido original del mensaje.

Tokens inexistentes

- Implementar un comportamiento explícito cuando el mensaje contenga tokens que no estén registrados en el repositorio.
- No se debe inventar información ni intentar reconstruir el valor original.

Propón el manejo de estos escenarios y utiliza una política consistente:

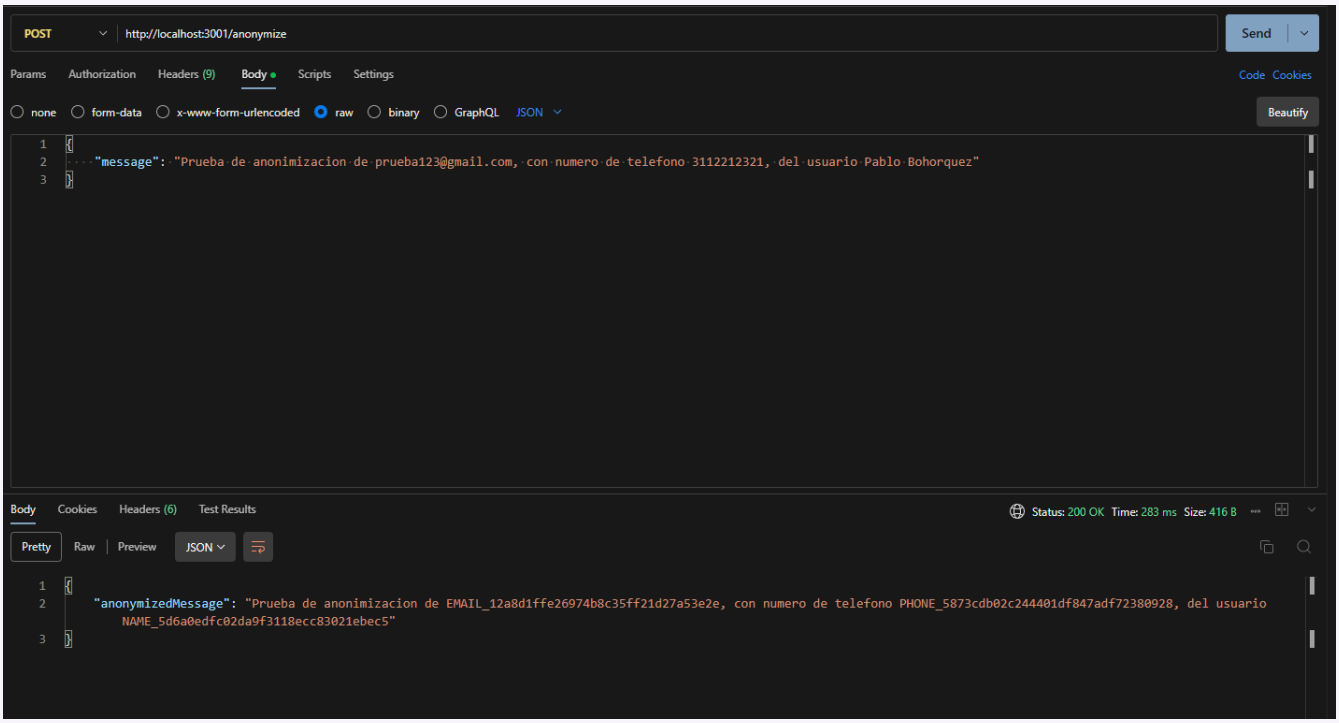
- Token válido: recuperar el valor original.
- Token desconocido: devolver un error controlado.
- Token malformado: aplicar las validaciones correspondientes.
- Mensaje sin tokens: devolver el mensaje sin modificaciones.

Para esta primera versión, prefiero que un mensaje con algún token desconocido no sea desanonimizado parcialmente. La operación debe fallar de forma controlada y no devolver información recuperada parcialmente.

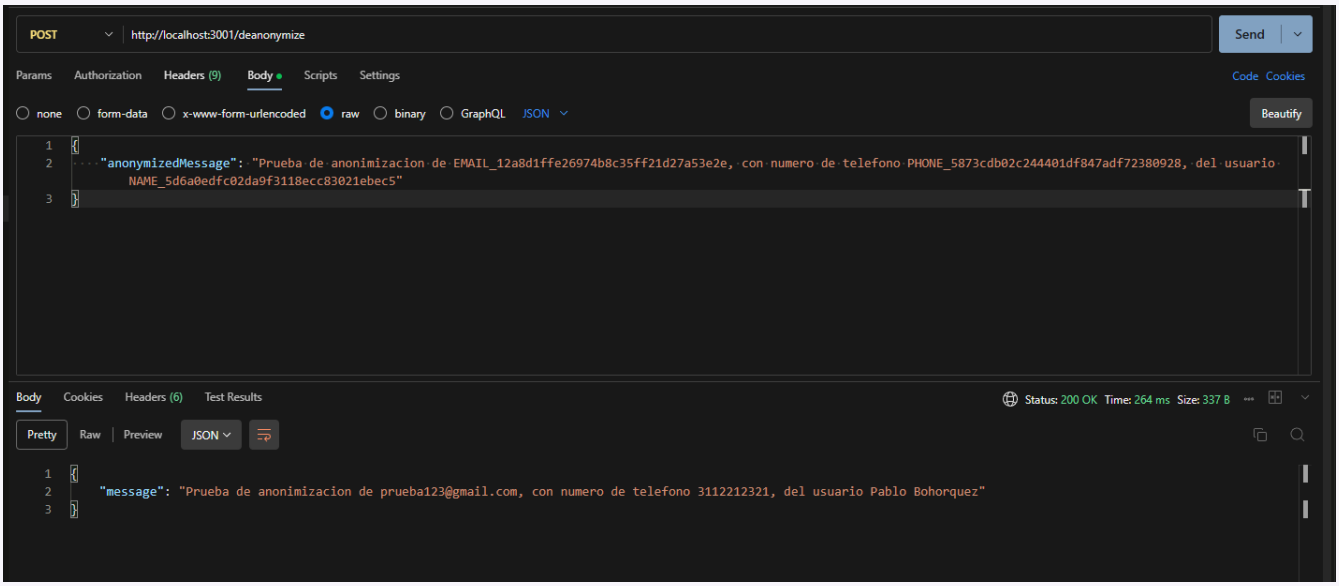
EVIDENCIA EN COMMIT DEL REPOSITORIO `ba6a57ea58d99237f0141492880e0509f1e5b98a` *(Implement deanonymization functionality and enhance pseudonymization process)*

EVIDENCIA DE FUNCIONAMIENTO

POST /anonymize



POST /deanonymize



TERCERA PARTE (CONEXION CON MONGO)

> Para este escenario si se debió hacer dos prompts para corregirlo, ya que la conexión la hizo local por Docker y no por cloud Atlas, el proceso fue el siguiente.

03 PRIMER PROMPT

Quiero incorporar persistencia utilizando MongoDB a nuestro proyecto Data Privacy Vault, que actualmente cuenta con los endpoints /anonymize y /deanonymize, ambos funcionando correctamente.

Objetivo

Reemplazar el repositorio en memoria actual por una implementación persistente en MongoDB, encargada de almacenar y recuperar las asociaciones entre los datos personales (PII) y sus respectivos tokens.

Requisitos

- Utilizar MongoDB como base de datos y Mongoose como ODM, salvo que exista una justificación técnica para utilizar el driver nativo.
- Implementar un modelo o esquema para almacenar:
  - Token generado.
  - Valor original asociado al token.
  - Tipo de información PII (NAME, EMAIL, PHONE, etc.).
  - Fecha de creación.
- Contexto o identificador del tenant, si ya está contemplado en la arquitectura.
- Implementar un repositorio MongoDB que cumpla con las interfaces existentes, respetando los principios de Clean Architecture y Dependency Inversion.
- Mantener desacoplada la lógica de negocio de MongoDB. Los casos de uso AnonymizeMessageUseCase y DeanonymizeMessageUseCase no deben conocer detalles de persistencia.
- Implementar índices y restricciones de unicidad para evitar duplicidad de tokens dentro de su ámbito correspondiente.
- Gestionar correctamente conexiones, errores, disponibilidad de la base de datos y configuración mediante variables de entorno.
- Garantizar que los datos originales almacenados estén protegidos mediante cifrado en reposo a nivel de aplicación, utilizando un mecanismo robusto como AES-256-GCM y una gestión segura de llaves. No almacenar llaves de cifrado directamente en el código ni en MongoDB.
- Evitar registrar información PII en logs o mensajes de error.

03 PRIMER PROMPT

Pruebas y criterios de aceptación

- Verificar que /anonymize almacene correctamente las asociaciones en MongoDB.
- Verificar que /deanonymize recupere los valores originales desde MongoDB.
- Comprobar que los datos persistan después de reiniciar la aplicación.
- Validar la unicidad de los tokens.
- Probar errores de conexión y persistencia.
- Verificar que los datos originales se almacenen cifrados.
- Mantener todas las pruebas existentes y evitar regresiones en ambos endpoints.

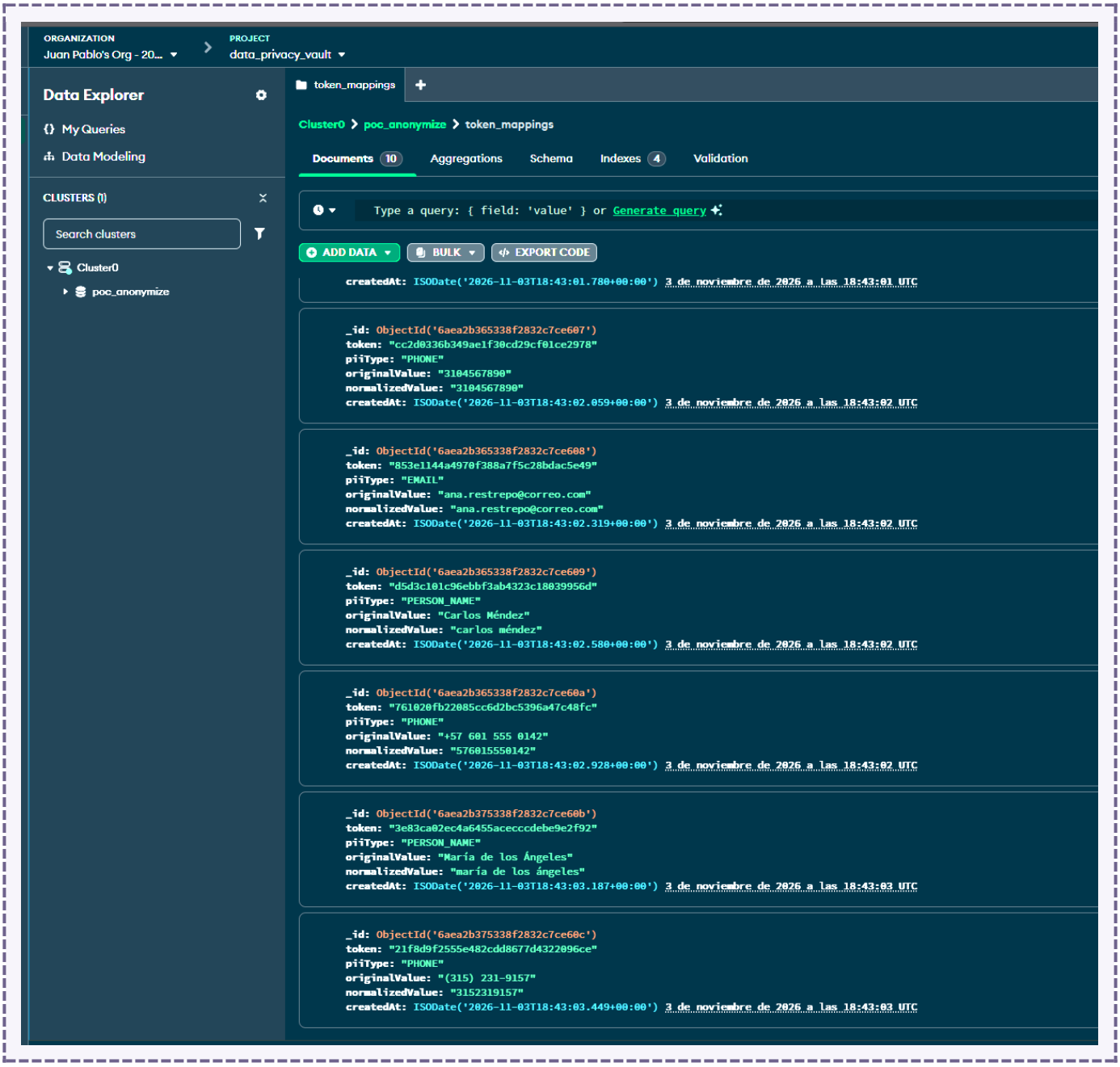
> **Importante: No modifiques innecesariamente los casos de uso ni los contratos existentes. Prioriza reutilizar la arquitectura actual e implementar MongoDB como un adaptador de infraestructura intercambiable.**

El resultado debe mantener la funcionalidad actual del Data Privacy Vault y permitir evolucionar posteriormente hacia una solución empresarial escalable y segura.

04 SEGUNDO PROMPT

Por favor hacer la implementación en la nube, ya tengo cuenta de MongoDB Atlas, si necesitas fijar alguna credencial o token dime donde estará y lo coloco.

EVIDENCIA EN COMMIT `3f26f4ab36f06e47718aa91aa3ef54c3f6845854` *(Add MongoDB integration for token mapping and enhance environment configuration)*



05 CUARTA Y ULTIMA PARTE (Integración con Open Ai y endpoint /secureChatGPT)

Quiero incorporar una integración con OpenAI a nuestro proyecto Data Privacy Vault y desarrollar un nuevo endpoint llamado POST /secureChatGPT.

1. Integración con OpenAI

Solicito crear una clase o servicio independiente para conectarse con la API de OpenAI utilizando el SDK oficial de Node.js.

05 CUARTA Y ULTIMA PARTE (Integración con Open Ai y endpoint /secureChatGPT)

Requisitos:

- Autenticación mediante API Key.
- La API Key debe configurarse mediante variables de entorno (OPENAI_API_KEY).
- Nunca incluir credenciales directamente en el código ni registrarlas en logs.
- Implementar un método para enviar mensajes a un modelo de OpenAI y obtener una respuesta de texto.
- Permitir configurar el modelo mediante variables de entorno.
- Implementar manejo de errores, timeouts y respuestas inesperadas.
- Mantener esta integración desacoplada de la lógica de negocio mediante una interfaz.

2. Nuevo endpoint: POST /secureChatGPT

El endpoint debe recibir un prompt que contenga información personal identificable (PII), procesarlo mediante nuestro Data Privacy Vault y enviarlo a OpenAI de forma protegida.

Ejemplo de petición

```
{
  "prompt": "Redacta un correo para Dago Borda, cuyo email es dborda@gmail.com y teléfono 3152319157, informándole sobre una entrevista de trabajo."
}
```

Flujo de procesamiento

Implementar el siguiente flujo:

- Recibir el prompt del cliente.
- Utilizar el caso de uso AnonymizeMessageUseCase existente para identificar y reemplazar los datos personales por sus respectivos tokens.
- Enviar exclusivamente el prompt anonimizado a OpenAI mediante el servicio de integración creado.
- Recibir la respuesta generada por OpenAI.
- Utilizar DeanonymizeMessageUseCase para reemplazar los tokens reconocidos por sus valores originales.
- Devolver al cliente la respuesta desanonimizada.

Importante: OpenAI nunca debe recibir los nombres, correos electrónicos, teléfonos u otros datos personales originales identificados por nuestro sistema.

Ejemplo de respuesta esperada

```
{
  "response": "Hola Dago Borda, queremos invitarte a una entrevista de trabajo. Puedes confirmar tu asistencia escribiendo a dborda@gmail.com o llamando al 3152319157."
}
```

05 CUARTA Y ULTIMA PARTE (Integración con Open Ai y endpoint /secureChatGPT)

Requisitos arquitectónicos

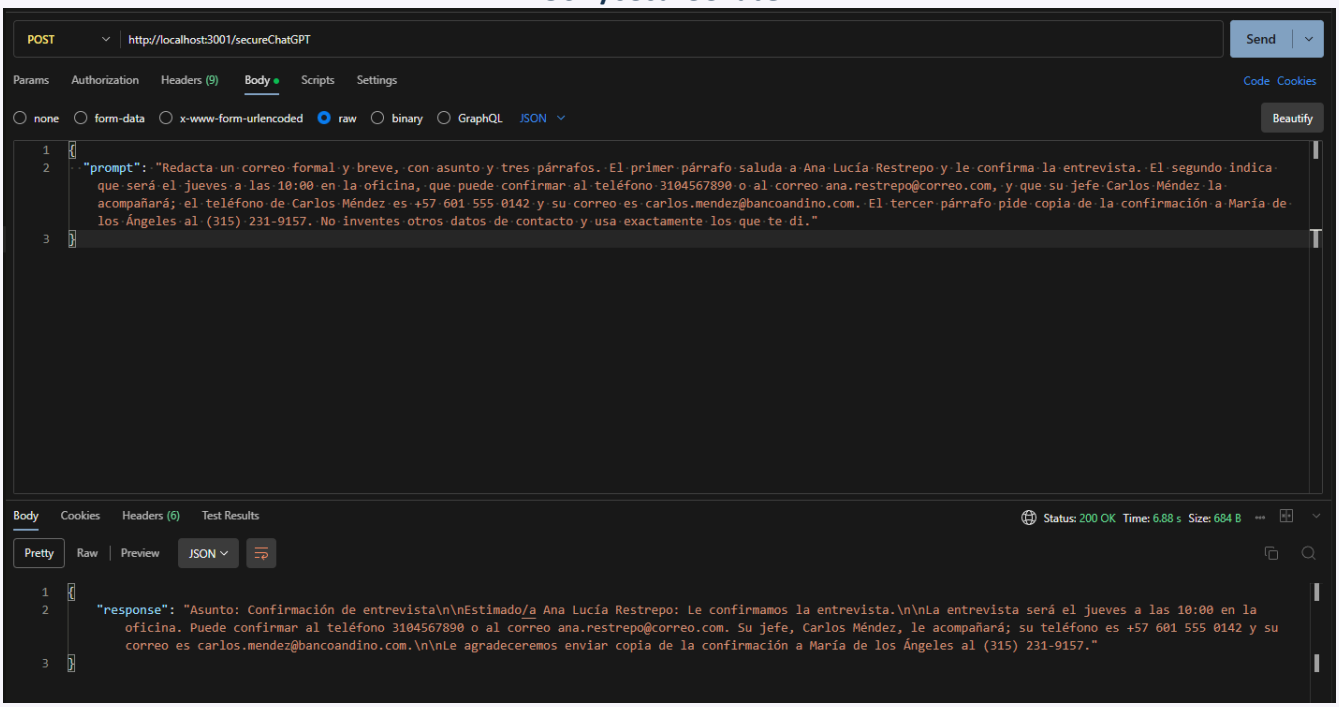
- Reutilizar los casos de uso de anonimización y desanonimización existentes.
- No duplicar lógica de detección, tokenización ni recuperación de información.
- Crear una interfaz para el proveedor de IA, evitando que los casos de uso dependan directamente del SDK de OpenAI.
- Implementar un nuevo caso de uso, por ejemplo SecureChatGPTUseCase, que orqueste todo el flujo.
- Mantener la integración con OpenAI dentro de Infrastructure.
- Mantener los controladores HTTP dentro de Presentation.
- Respetar Clean Architecture, SOLID, Dependency Inversion y Separation of Concerns.

4. Consideraciones de seguridad y confiabilidad

- No enviar información PII original a OpenAI.
- No registrar prompts originales ni respuestas que contengan información personal.
- Proteger la API Key mediante variables de entorno.
- Implementar validaciones de entrada y límites de tamaño.
- Manejar errores de conexión, timeouts, límites de consumo y respuestas fallidas de OpenAI.
- No devolver respuestas parcialmente desanonimizadas.
- Si OpenAI devuelve tokens inexistentes, alterados o desconocidos, aplicar un manejo de errores controlado y no revelar información incorrecta.
- Garantizar que las operaciones de desanonimización respeten los mecanismos de autorización y aislamiento de datos existentes.
- Ten presente que los modelos de IA pueden modificar, omitir o generar tokens. No asumas que siempre conservarán exactamente los identificadores recibidos. Implementa validaciones para evitar sustituciones incorrectas.

EVIDENCIA EN COMMIT `2c9013e87db33fcc3d3a57c818795c2f7252e437` *(Integrate OpenAI API for secure chat functionality)*

POST /secureChatGPT



CONCLUSIONES Y REFLEXIONES

1. Viabilidad de la solución tecnológica

El desarrollo de la prueba de concepto permitió demostrar la viabilidad técnica de integrar mecanismos de protección de información personal con servicios de inteligencia artificial generativa. Mediante los endpoints y casos de uso implementados, se logró construir un flujo de seudonimización, persistencia y recuperación de información, evidenciando cómo diferentes tecnologías pueden integrarse para resolver una necesidad concreta de protección de datos.

2. Aprendizajes del desarrollo asistido por inteligencia artificial

El uso de Cursor como asistente de programación permitió avanzar progresivamente en la implementación de la solución. Sin embargo, el ejercicio también evidenció que la generación de código mediante IA requiere supervisión, validación y ajustes por parte del equipo. Un ejemplo fue la necesidad de precisar los requerimientos para utilizar MongoDB Atlas en lugar de una instancia local. Esto permitió comprender que la calidad de los resultados depende tanto de la claridad de los prompts como del criterio técnico utilizado para evaluar las soluciones propuestas.

3. Protección de datos y uso responsable de la IA

Uno de los principales aprendizajes fue comprender la diferencia entre anonimización y seudonimización. La solución sustituye información personal por tokens, manteniendo su correspondencia para permitir una recuperación posterior. Esta estrategia reduce la exposición de los datos identificados durante su envío a OpenAI, pero no garantiza una protección absoluta. La efectividad depende de la capacidad de detección del sistema y de la seguridad con la que se administran las asociaciones almacenadas.

4. Importancia de la arquitectura y la integración tecnológica

La aplicación de principios de Clean Architecture y separación de responsabilidades facilitó incorporar nuevas funcionalidades sin reconstruir los componentes existentes. La evolución desde los endpoints iniciales hasta la integración con MongoDB Atlas y OpenAI evidenció las ventajas de diseñar soluciones modulares y desacopladas, particularmente cuando se requiere integrar distintos servicios y proveedores tecnológicos.

En la actualidad, el uso de la inteligencia artificial suele priorizar el resultado funcional inmediato, dejando de lado una arquitectura de software alineada con las necesidades específicas de cada organización. Para lograr un desarrollo verdaderamente eficiente y sostenible a largo plazo, no se debe delegar todo el proceso en la IA; el diseño estructural y estratégico sigue requiriendo del criterio humano.

5. Limitaciones identificadas y oportunidades de mejora

Aunque la prueba de concepto cumple su propósito de demostrar el funcionamiento del flujo principal, se identificaron aspectos que deben fortalecerse antes de considerar su implementación en un entorno productivo. Entre ellos se encuentran la precisión de la detección de información personal mediante expresiones regulares, el cifrado de los datos originales almacenados, los mecanismos de autorización y el aislamiento de información entre usuarios o clientes. Estas limitaciones permiten establecer una ruta de mejora fundamentada en los riesgos identificados durante el desarrollo.

6. Relación con los objetivos del curso y evolución hacia un MVP

El taller permitió relacionar los conceptos de integración tecnológica y experimentación con una implementación funcional. Desde una perspectiva de negocio, la prueba de concepto constituye un primer paso para evaluar cómo una organización del sector financiero podría incorporar servicios de IA generativa reduciendo la exposición de información sensible. No obstante, avanzar hacia un producto mínimo viable requeriría validar indicadores de precisión en la detección de datos, tiempos de respuesta, costos operativos y controles de seguridad. Esto demuestra que comprobar la viabilidad técnica es una etapa necesaria, pero no suficiente, para determinar la viabilidad de una iniciativa empresarial.

7. Importancia de la ingeniería de prompts

La experiencia permitió comprender que la ingeniería de prompts es una habilidad cada vez más relevante para el desarrollo de software. No se trata únicamente de solicitar a una herramienta de inteligencia artificial que genere código, sino de proporcionar instrucciones claras, estructuradas y contextualizadas que definan correctamente los objetivos, requisitos funcionales, restricciones técnicas y criterios de calidad del proyecto.

Como punto de partida, se realizó un análisis del prompt proporcionado en la guía de la actividad, identificando oportunidades de mejora para obtener un resultado más completo y acorde con las necesidades del proyecto de la materia. A partir de este ejercicio, se reformularon las instrucciones, incorporando aspectos como arquitectura limpia, separación de responsabilidades, uso de interfaces, principios SOLID, escalabilidad y buenas prácticas de seguridad.

Este proceso permitió pasar de una solicitud inicial relativamente general a una especificación más estructurada, que orientara a la inteligencia artificial no solo en la generación de código, sino también en las decisiones arquitectónicas y técnicas necesarias para construir una solución mantenible, segura y preparada para evolucionar, tal como se pudo ver en los diferentes prompts que construimos en el taller.